

Homework 3

Problem 1

a)

$$\hat{\beta}_{ridge} = \operatorname{argmin}_{\beta} \sum_{j=1}^p \left[((X^T Y)_j - \beta_j)^2 + \lambda \beta_j^2 \right]$$

$$\hat{\beta}_{lasso} = \operatorname{argmin}_{\beta} \sum_{j=1}^p \left[((X^T Y)_j - \beta_j)^2 + \lambda |\beta_j| \right]$$

b)

For convenience, define $z_j = (X^T Y)_j$.

Ridge regression: $\hat{\beta}_j = \operatorname{argmin}_{\beta} (z_j - \beta_j)^2 + \lambda \beta_j^2$. Taking the derivative of the objective and setting it equal to zero yields

$$-2(z_j - \beta_j) + 2\lambda \beta_j = 0.$$

Solving for β_j gives

$$\hat{\beta}_j = \frac{1}{1 + \lambda} z_j = \frac{1}{1 + \lambda} (X^T Y)_j.$$

Therefore, in this orthogonal case, $\hat{\beta}_j$ is just the OLS coefficient reduced by a multiplicative factor.

Lasso regression: $\hat{\beta}_j = \operatorname{argmin}_{\beta} (z_j - \beta_j)^2 + \lambda |\beta_j|$. The absolute value means we don't want to try to just take a derivative. Instead, we'll consider the two halves of the objective separately: $\beta_j \geq 0$ (call it RHS), $\beta_j \leq 0$ (call it LHS). On the two halves, the objective function can be written as:

$$\text{objective} = \begin{cases} (z_j - \beta_j)^2 + \lambda \beta_j & \text{RHS} \\ (z_j - \beta_j)^2 - \lambda \beta_j & \text{LHS} \end{cases}$$

Each of these functions separately is just a quadratic. We can differentiate and solve to find the global minimizers of each quadratic, though they are not guaranteed to fall on the correct half of the β_j space. The global minimizers are:

$$\text{RHS: } \beta_j = z_j - \frac{\lambda}{2}$$

$$\text{LHS: } \beta_j = z_j + \frac{\lambda}{2}$$

However, when the minimizer does not fall in the correct β_j set, the minimizer over that set is at $\beta_j = 0$. We can write the actual minimizer for each side as

$$\beta_j = \begin{cases} z_j - \lambda/2 & \text{if } z_j > \lambda/2 \\ 0 & \text{if } z_j \leq \lambda/2 \end{cases} \quad \text{RHS}$$

$$\beta_j = \begin{cases} z_j + \lambda/2 & \text{if } z_j < -\lambda/2 \\ 0 & \text{if } z_j \geq -\lambda/2 \end{cases} \quad \text{LHS}$$

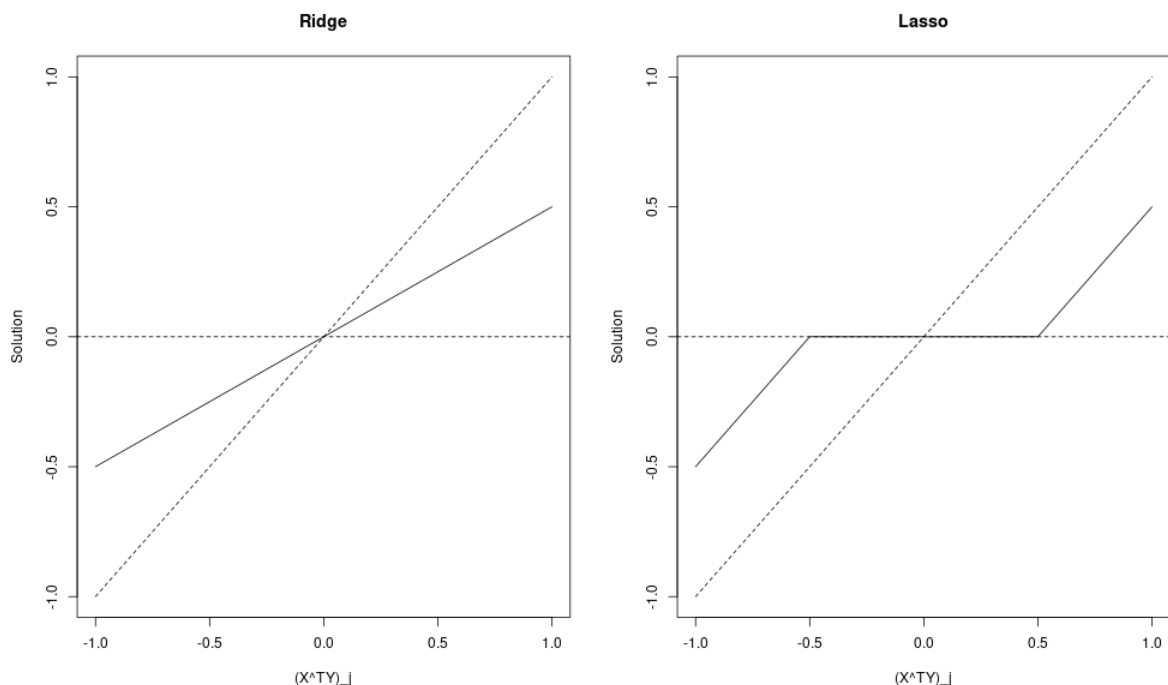
Putting these together, we obtain our final solution for the lasso:

$$\hat{\beta}_j = \begin{cases} (X^T Y)_j - \lambda/2 & \text{if } (X^T Y)_j > \lambda/2 \\ (X^T Y)_j + \lambda/2 & \text{if } (X^T Y)_j < -\lambda/2 \\ 0 & \text{otherwise} \end{cases}$$

This is sometimes written as

$$\hat{\beta}_j = \left(|(X^T Y)_j| - \frac{\lambda}{2} \right)_+ \text{sign}(|(X^T Y)_j|)$$

c and d)



We see from the solutions and the plots that ridge regression corresponds to a reduction of OLS by a multiplicative constant (so it impacts large coefficients more in an absolute sense), while

lasso corresponds to an additive shift toward zero (so it impacts small coefficients more in a relative sense)

Problem 2

```
In [1]: import numpy as np
import pandas as pd

import scipy.stats

from sklearn import linear_model
import statsmodels.api as sm
import statsmodels.formula.api as smf

from progressbar import ProgressBar

import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: scipy.random.seed(1)
```

```
In [3]: def generate_data_problem1(size=20, p=1):
        """
        size: number of observations
        p: number of dimensions of X
        """
        mvnorm = scipy.stats.multivariate_normal(mean=[0]*p, cov=np.diag(np.ones(
        X = mvnorm.rvs(size=size).reshape(size, p)
        y = 4*X[:,0] + scipy.stats.norm(scale=1).rvs(size=size)

        return X, y
```

a)

```
In [4]: p_range = np.arange(79) + 2
n = 100
n_sim = 100

mse_mat = np.zeros((p_range.shape[0], n))

bar = ProgressBar()
for p_count, p in bar(list(enumerate(p_range))):
    # ^this is kinda ugly (feel free to remove bar(list()))
    for sim in np.arange(n_sim):
        X_train, y_train = generate_data_problem1(size=n, p=p)

        X_test, y_test = generate_data_problem1(size=n, p=p)
        lm_out = sm.OLS(y_train, X_train, hasconst=True).fit()
        # ^ in this setup (with `hasconst = True`), it makes it so that
        # no beta_0 is fit (the working of the problem could be read either w
        y_pred_test = lm_out.predict(exog=X_test)

        mse_mat[p_count, sim] = np.mean((y_test - y_pred_test)**2)
```

100% (79 of 79) |#####| Elapsed Time: 0:01:02 Time: 0:01:02

b)

```
In [5]: mse_mat_lasso = np.zeros((p_range.shape[0], n))

bar = ProgressBar()
for p_count, p in bar(list(enumerate(p_range))):
    # ^this is kinda ugly (feel free to remove bar(list()))
    for sim in np.arange(n_sim):
        X_train, y_train = generate_data_problem1(size=n, p=p)

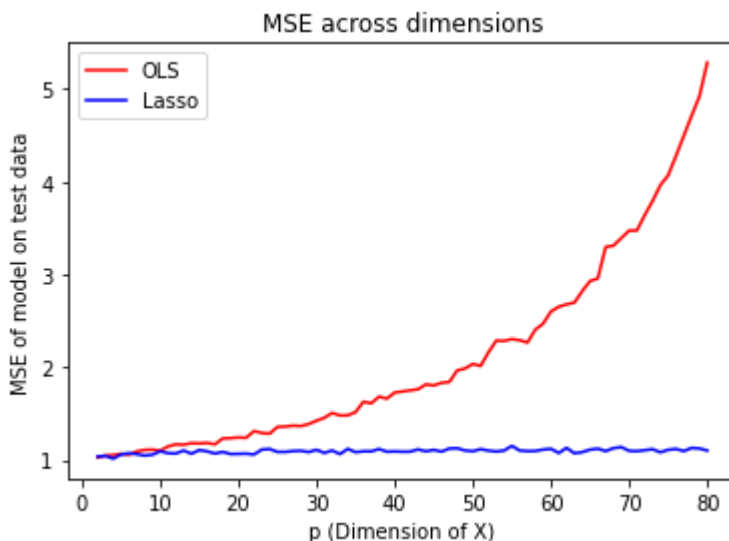
        X_test, y_test = generate_data_problem1(size=n, p=p)
        lm_lasso = linear_model.Lasso(alpha=np.sqrt(np.log(p)/(50*n)),
                                      normalize=True, fit_intercept=True)
        lm_lasso.fit(X_train, y_train)
        # ^ in this setup (with `hasconst = True`), it makes it so that
        # no beta_0 is fit (the working of the problem could be read either w
        y_pred_test = lm_lasso.predict(X_test)

        mse_mat_lasso[p_count, sim] = np.mean((y_test - y_pred_test)**2)
```

100% (79 of 79) |#####| Elapsed Time: 0:00:59 Time: 0:00:59

```
In [6]: mean_pred_errors = np.mean(mse_mat, axis = 1)
mean_pred_errors_lasso = np.mean(mse_mat_lasso, axis = 1)
```

```
In [7]: fig, ax = plt.subplots()
ax.set(xlabel="p (Dimension of X)",
      ylabel="MSE of model on test data",
      title="MSE across dimensions")
ax.plot(p_range, mean_pred_errors, "k-", color="red", label="OLS")
ax.plot(p_range, mean_pred_errors_lasso, "k-", color="blue", label="Lasso")
plt.legend();
```



The out of sample MSE for OLS increases with the number of junk covariates, but grows very

very slowly for the Lasso.

Problem 3

```
In [8]: import rpy2
        %load_ext rpy2.ipython

        import warnings
        warnings.filterwarnings("ignore")
```

```
In [9]: %%R
        library(glmnet)

        train_data = read.csv("trends_train.csv")
        test_data = read.csv("trends_test.csv")

        X_train = as.matrix(train_data[, 2:(ncol(train_data)-1)])
        Y_train = as.matrix(train_data[, ncol(train_data)])

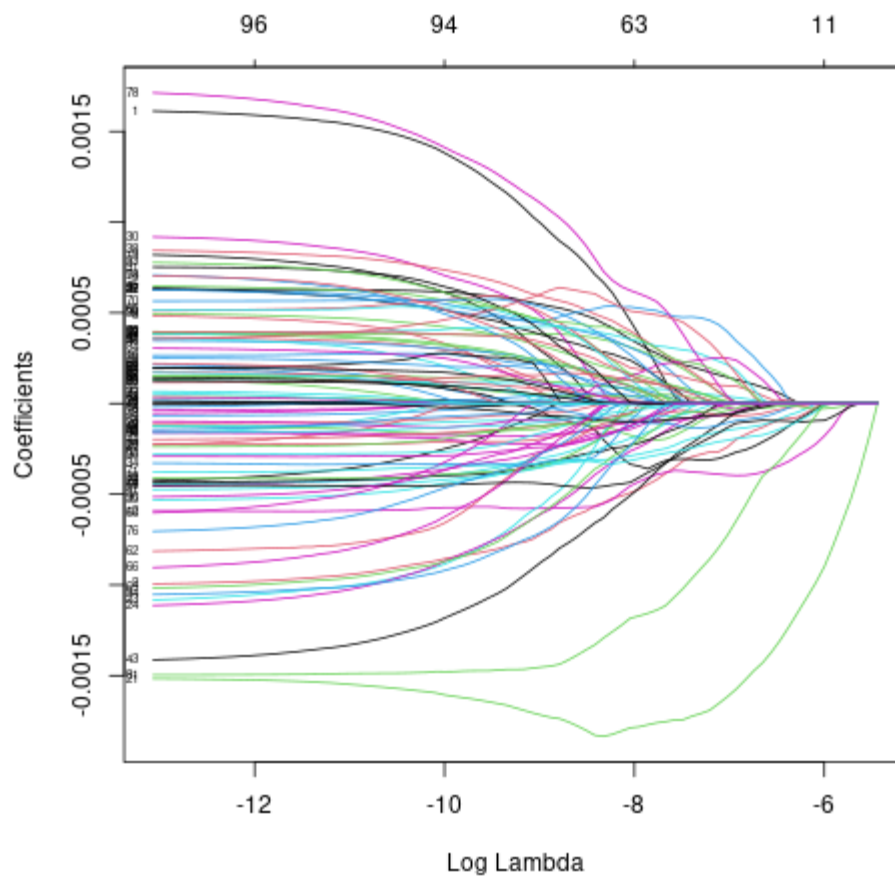
        X_test = as.matrix(test_data[, 2:(ncol(test_data)-1)])
        Y_test = as.matrix(test_data[, ncol(test_data)])
```

R[write to console]: Loading required package: Matrix

R[write to console]: Loaded glmnet 4.0-2

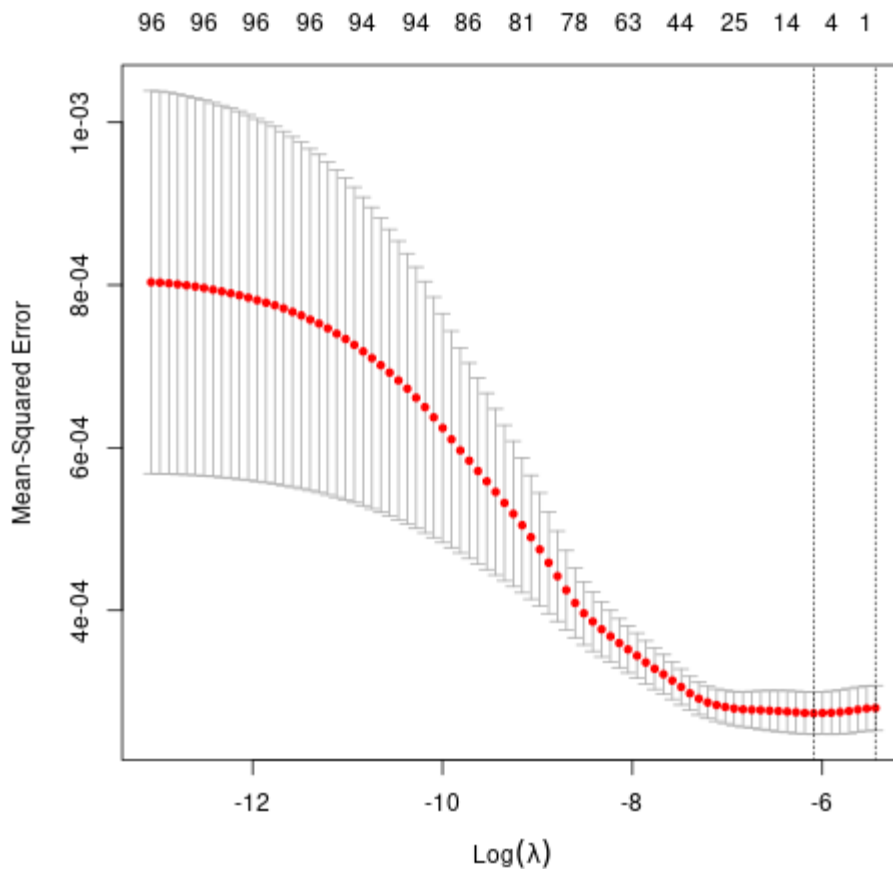
a).

```
In [10]: %%R
        lasso_out = glmnet(X_train, Y_train)
        plot(lasso_out, xvar = "lambda", label = TRUE)
```



b).

```
In [11]: %%R
set.seed(0)
lasso_cv = cv.glmnet(X_train, Y_train)
plot(lasso_cv, lwd=2)
```



c).

```
In [12]: %R
coef_lse = coef.glmnet(lasso_out, s = lasso_cv$lambda.1se)
coef_min = coef.glmnet(lasso_out, s = lasso_cv$lambda.min)

coef_lse_nonzero = as.matrix(coef_lse[which(coef_lse != 0)])
rownames(coef_lse_nonzero) = rownames(coef_lse)[which(coef_lse != 0)]
colnames(coef_lse_nonzero) = "Coefficient"

coef_min_nonzero = as.matrix(coef_min[which(coef_min != 0)])
rownames(coef_min_nonzero) = rownames(coef_min)[which(coef_min != 0)]
colnames(coef_min_nonzero) = "Coefficient"

print("The Nonzero Coefficient for 1 s.e. Rule")
print(coef_lse_nonzero)
print("The Nonzero Coefficient for Minimization Rule")
print(coef_min_nonzero)
```

```
[1] "The Nonzero Coefficient for 1 s.e. Rule"
      Coefficient
(Intercept) 0.001572434
[1] "The Nonzero Coefficient for Minimization Rule"
      Coefficient
(Intercept) 1.643469e-03
growth      -2.004550e-05
dow         -3.153716e-05
```

```

stocks      -5.304563e-06
invest      -2.240417e-04
leverage    -4.487841e-06
cash        -9.892606e-04
returns     -2.098079e-06
tourism     -9.486248e-05
health      -3.147563e-05
fine        -4.343406e-05

```

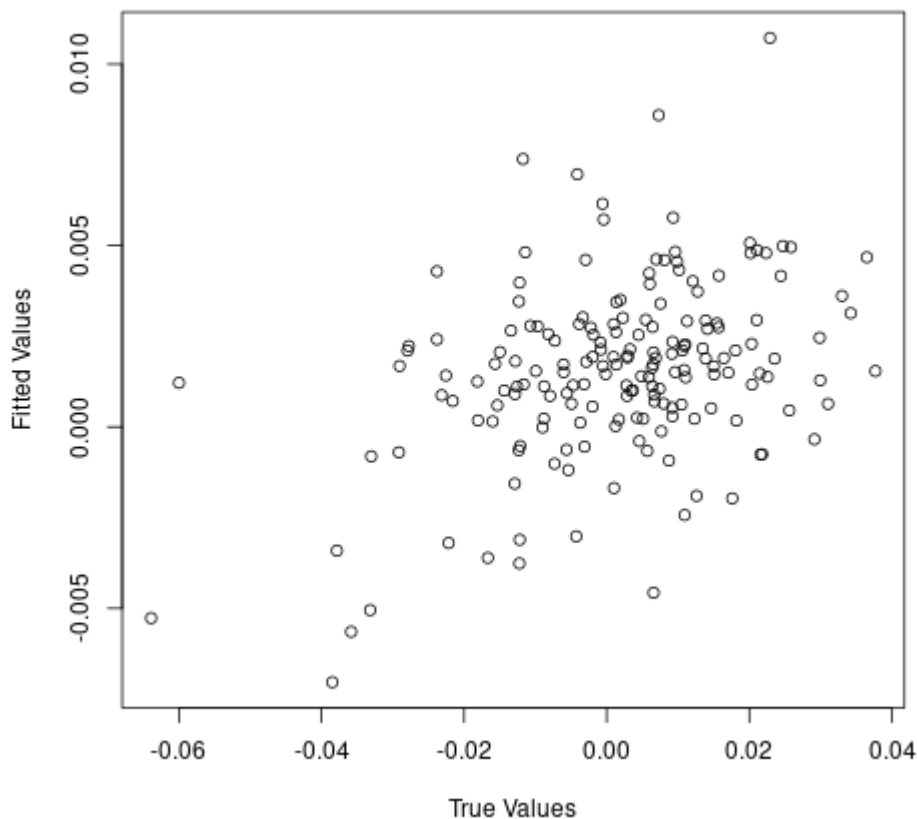
There are very few signals. In particular, the 1 s.e. rule picks the empty model! Generally, this is a hint that there is little or no useful (linear) signal. However, it's tempting to optimistically switch and use lambda min in these cases because you get a non-empty model, and it looks good on the training data.

d).

```

In [13]: %%R
fitted_train = predict.glmnet(lasso_out, newx = X_train, s = lasso_cv$lambda.
plot(Y_train, fitted_train, type = "p", xlab = "True Values", ylab = "Fitted

```



There is a positive linear pattern between the fitted value and true value.

e).


```
In [14]: %%R
sign_train = (fitted_train > 0) * 2 - 1
Rlog_trade = t(sign_train) %% Y_train
Rlog_hold = sum(Y_train)

R_trade = exp(Rlog_trade) - 1
R_hold = exp(Rlog_hold) - 1
R_ratio = R_trade / R_hold

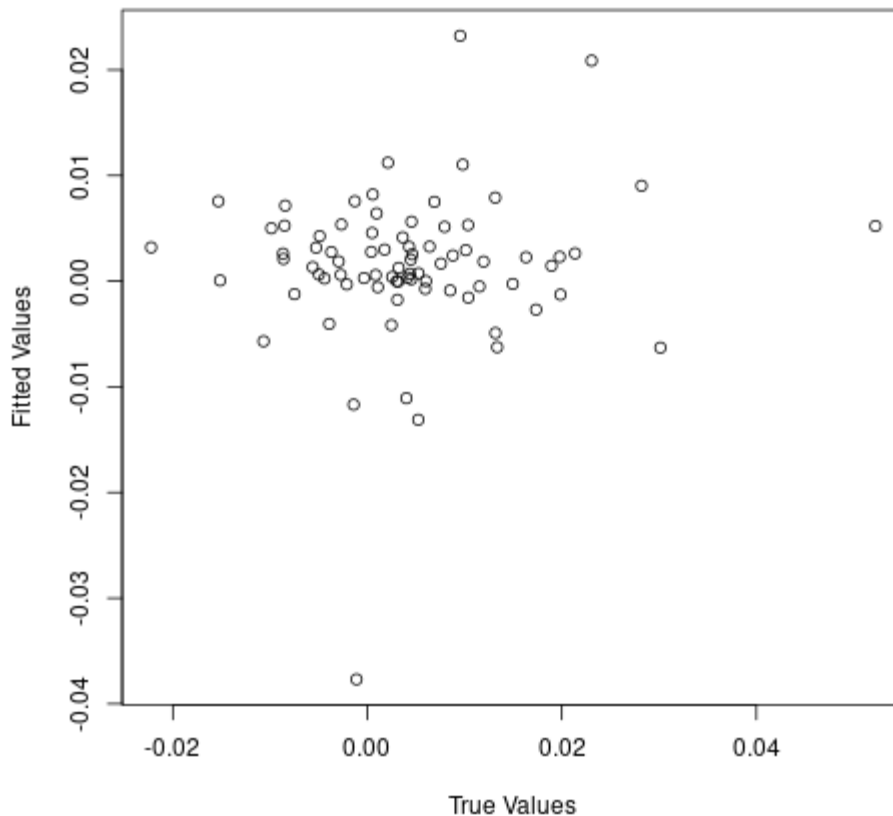
print("The return rate for trading strategy:")
print(as.numeric(R_trade))
print("The return rate for holding strategy:")
print(as.numeric(R_hold))
print("The ratio of return rate for the trading and return rate for holding:")
print(as.numeric(R_ratio))

[1] "The return rate for trading strategy:"
[1] 1.22705
[1] "The return rate for holding strategy:"
[1] 0.3271557
[1] "The ratio of return rate for the trading and return rate for holding:"
[1] 3.750661
```

The trading strategy outperforms the holding strategy by 354% (on the training data).

f).

```
In [15]: %%R
fitted_test = predict.glmnet(lasso_out, newx = X_test, s = lasso_cv$lambda.mi
plot(Y_test, fitted_test, type = "p", xlab = "True Values", ylab = "Fitted Va
```



```
In [16]: %%R
sign_test = (fitted_test > 0) * 2 - 1
Rlog_trade_test = t(sign_test) %% Y_test
Rlog_hold_test = sum(Y_test)

R_trade_test = exp(Rlog_trade_test) - 1
R_hold_test = exp(Rlog_hold_test) - 1
R_ratio_test = R_trade_test / R_hold_test

print("The return rate for trading strategy:")
print(as.numeric(R_trade_test))
print("The return rate for holding strategy:")
print(as.numeric(R_hold_test))
print("The ratio of return rate for the trading and return rate for holding:")
print(as.numeric(R_ratio_test))

[1] "The return rate for trading strategy:"
[1] 0.06715828
[1] "The return rate for holding strategy:"
[1] 0.4136197
[1] "The ratio of return rate for the trading and return rate for holding:"
[1] 0.1623672
```

The trading strategy only earns as much as 23.38% of the holding strategy on test datas. Thus trading based on google trends does not seem to be a good strategy. The 1 s.e. rule would have helped us realize this by selecting the empty model, which is the same as the holding strategy.